# pycomm3_slc_cli

## User Manual

Application version 0.3.0
Document revision 1 — November 10, 2026
Append Automation

*A command line tool for reading and writing Allen-Bradley SLC 500 and MicroLogix data files over EtherNet/IP, on any Windows PC, with no Rockwell software and no Python installation.*

**WARNING** This application writes to programmable logic controllers that control industrial machinery. Writing a value to a controller that is running equipment can start a motor, open a valve or change a setpoint without warning, and can injure people or damage equipment. Read Section 2 before you use the Write or SetPLCTime commands. Use of this application is entirely at your own risk.

# 1. Introduction

## 1.1 Purpose of this manual

This manual describes how to install and operate pycomm3_slc_cli, what each of its commands does, and the hazards that come with using it. It is written for the person who has to get a value out of, or into, an Allen-Bradley SLC 500 family controller and does not have Rockwell programming software in front of them.

## 1.2 What the application is

pycomm3_slc_cli is a single Windows executable that wraps the SLCDriver of the open source pycomm3 library. It opens an EtherNet/IP connection to a controller, carries out one or more commands, and prints the result to the console. There is no installer, no service, no configuration file and no graphical interface. Everything the application does is driven from the command line or from its own interactive prompt.

Typical uses are reading a production counter or a recipe setpoint, correcting a controller clock that has drifted, capturing a block of data file values to a text file for a report, and finding out why a controller has faulted.

## 1.3 Supported controllers

The application talks to controllers that use the PCCC command set over EtherNet/IP. In practice that means the SLC 5/05, the MicroLogix family, and SLC 5/03 and SLC 5/04 processors reached through an Ethernet interface module or a gateway. It does not talk to ControlLogix or CompactLogix controllers, which use symbolic tags rather than data files.

Some information the application reports is specific to the processor. The last major error fault code in S:54, for example, exists only on SLC 5/03 and higher processors.

## 1.4 Conventions used in this manual

- Text in a fixed pitch font is typed exactly as shown, or is output printed by the application.
- Angle brackets mark a value you supply, as in Read <tag>. Square brackets mark an optional argument.
- The prompt pycomm3_slc_cli> is the application's own prompt. The prompt C:\pycomm3_demo> is the Windows command prompt.
- Every screenshot and every console listing in this manual was captured from a real session against a real controller. Section 3.5 describes that equipment.

## 2. Safety and warnings

The hazards below are a property of writing to a live industrial controller, not of any defect in this application. Read all of them before you use the Write or SetPLCTime commands.

### 2.1 Writing to a controller that is running equipment

> **WARNING** A controller does not distinguish between a value written by its own program and a value written over the network. If you write to an address the running program acts on, the machine responds immediately. There is no confirmation prompt, no undo and no dry run. Assume every Write takes effect before you press Enter.

Before writing to a controller that is powered and in Run mode, confirm that you know what the address is used for, that the value you are about to write is within the range the program expects, and that nobody is working on the equipment. Where the work allows it, put the machine in a safe state first.

### 2.2 Confirm the target address before every write

The IP address is supplied on the command line or with the IPAddress command, and the application connects to whatever answers at that address. A typed digit in the wrong place can connect you to a different machine on the same plant network. Read a known value back from the controller first to confirm you are talking to the one you intended.

### 2.3 The application does not understand your process

pycomm3_slc_cli validates that a value fits the data file type it is being written to. It has no knowledge of engineering units, scaling, interlocks or valid ranges. A value that is correct for the data type can still be badly wrong for the machine.

### 2.4 Setting the controller clock

> **WARNING** SetPLCTime overwrites the controller clock with the clock of the PC that runs the command. If the PC clock is wrong, the controller clock becomes wrong. If the controller program uses its clock to timestamp production records, start a shift or trigger a scheduled action, changing the clock changes the behavior of that program.

The six clock words are written one at a time rather than as a single block, so a program that reads the clock while the command is running can briefly see a mixture of the old and the new time. The application writes only the clock and calendar words S:37 through S:42; it does not update any other time related word.

### 2.5 Network exposure

EtherNet/IP carries no authentication and no encryption. Anyone who can reach the controller on the network can read and write its data files with this application or any other. Running this tool does not create that exposure, but it does make use of it. Keep controllers on a segregated network, and do not leave a PC with this application on it connected to both the plant network and an untrusted network.

### 2.6 Fault code information is advisory

The fault descriptions printed by GetFaultCodes are transcribed from Rockwell publication 1747-RM001G-EN-P. They are an aid to diagnosis, not a substitute for it. Confirm anything you are going to act on against the manual for the processor in front of you.

## 3. Installation and requirements

### 3.1 What you need

- A PC running 64-bit Windows 7 SP1 or any later version of Windows. The executable is 64-bit and will not run on 32-bit Windows.
- A network path from that PC to the controller.
- Nothing else. Rockwell software, the RSLinx driver and a Python installation are all unnecessary.

### 3.2 Installing

There is no installer. Copy pycomm3_slc_cli.exe to any folder you can write to and run it from a command prompt. A folder of its own is convenient, because the multi-tag commands read and write text files in the current directory. The examples in this manual use C:\pycomm3_demo.

```
C:\> mkdir C:\pycomm3_demo
C:\> copy pycomm3_slc_cli.exe C:\pycomm3_demo
C:\> cd \pycomm3_demo
```

> **Note** The executable is built with PyInstaller, which unpacks itself to a temporary folder on each run. The first start after a copy is usually a second or two slower than later ones, and some endpoint protection products inspect the unpacked files.

### 3.3 Network requirements

The application opens a TCP connection to port 44818 on the controller, which is the registered EtherNet/IP port. The PC must be able to route to the controller address and the Windows firewall must allow the outbound connection. If a connection cannot be opened the application reports the reason and exits with code 1.

### 3.4 Building the executable from source

The executable is produced with PyInstaller against Python 3.8.10 on Windows 7 SP1. Python 3.8 is the last release with official Windows 7 support, so building on Windows 7 is what keeps the result usable on Windows 7 and on every later Windows.

```
git clone https://github.com/AppendAutomation/pycomm3_slc_cli.git
cd pycomm3_slc_cli
pip install pycomm3
pip install pyinstaller
pyinstaller -F pycomm3_slc_cli.py
```

The finished executable is written to the dist folder.

### 3.5 The equipment used for the examples

Every screenshot and listing in this manual was captured from the session described below. Values read from the controller change from one figure to the next because the controller program was running throughout.

**Table 1 — Equipment used to capture the examples in this manual**

| Item | Detail |
|---|---|
| **Test PC** | Windows 7 Professional SP1, 64-bit, build 6.1.7601 |
| **Application** | pycomm3_slc_cli.exe 0.3.0, run from C:\pycomm3_demo |
| **Controller** | Allen-Bradley SLC 500 family processor at 192.168.2.150 |
| **Connection** | EtherNet/IP over the local Ethernet network |
| **Captured** | November 10, 2026 |

## 4. Running the application

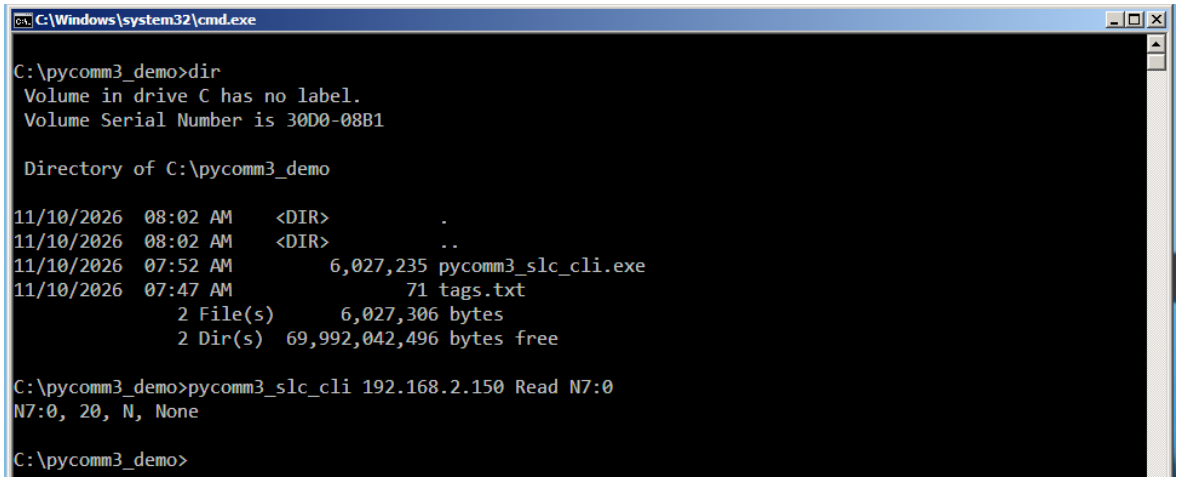
### 4.1 Three ways to start it

The application reads its first argument to decide how to run. Anything with four dot separated parts is treated as the address of the controller; anything else is treated as the first word of a command.

**One command at a time.** Give it an address and a command, and it connects, runs the one command and exits. This is the form to use in a batch file or a scheduled task.

```
C:\pycomm3_demo>pycomm3_slc_cli 192.168.2.150 Read N7:0
N7:0, 20, N, None
```

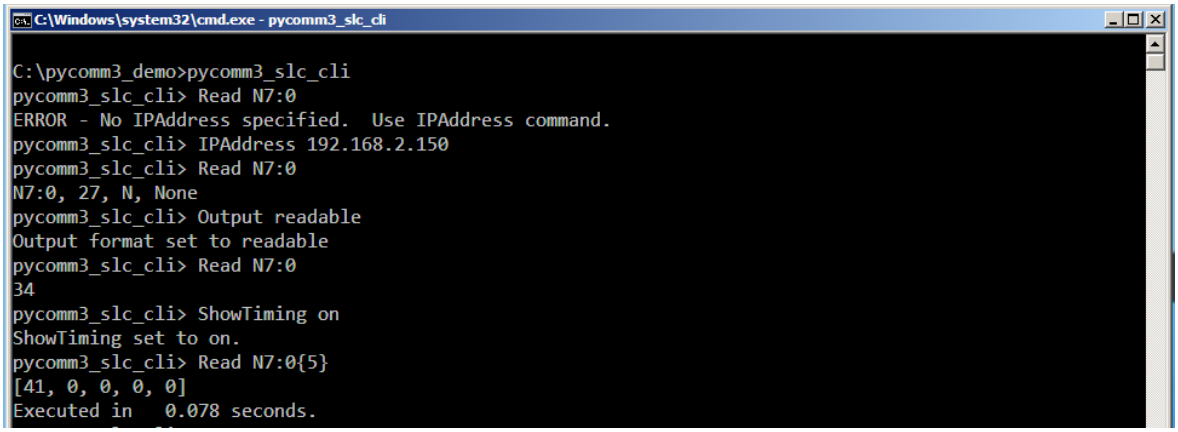
**Figure 1 — Running a single command against the controller at 192.168.2.150**




**A console session on a known controller.** Give it an address on its own and it connects, then presents its own prompt. The connection stays open until you type Quit, so a series of commands runs far faster than the same commands run one at a time.

**A console session with no controller yet.** Start it with no arguments at all and it presents its prompt without a connection. Commands that need the controller report that no address has been set until you supply one with the IPAddress command.

**Figure 2 — A console session: the error before an address is set, the IPAddress command, and the effect of Output and ShowTiming**




## 4.2 Command line syntax

```
pycomm3_slc_cli [<ip address>] [<command> [<arguments>]]
```

**Table 2 — How the first argument is interpreted**

| First argument | What happens |
|---|---|
| **An address, with a command after it** | Connects, runs that one command, closes and exits. |
| **An address on its own** | Connects and opens the interactive prompt. |
| **A command word** | Opens the interactive prompt with no connection. Only commands that do not need the controller, such as Help and OptimizeTagFile, will do anything useful. |
| **Nothing** | Opens the interactive prompt with no connection. |

> **Note** Only a numeric address in dotted form is reliable. The test is simply that the argument has four parts separated by dots, so a host name of that shape is passed through to the network layer and may or may not resolve. Use the IP address.

## 4.3 Case sensitivity

Command names are not case sensitive: Read, read and READ are the same command. Everything after the command name is passed through exactly as you typed it, which matters in three places.

- Tag names keep their case and are echoed back as typed.
- File names keep their case. Windows itself does not distinguish case in file names, but the name is used as given.
- The arguments to Output and ShowTiming must be lower case. Output readable works; Output Readable is rejected as an invalid format. The same applies to ShowTiming on and off.

### 4.4 Exit codes

Exit codes matter when the application is called from a batch file. Only a failure to connect is reported as a failure.

**Table 3 — Exit codes returned to the calling process**

| Code | Meaning |
|---|---|
| **0** | The application ran. A command that printed an error message, including a read of an address that does not exist, still exits with 0. |
| **1** | An address and a command were supplied but the connection to the controller could not be opened. |

## 5. Addressing SLC data files

An SLC 500 stores its data in numbered files, each holding elements of one type. An address names the file, the element within it and, where it applies, a bit or a field of a structured element. The application passes the address you type straight through to the controller, so any address RSLogix 500 accepts is accepted here.

### 5.1 Address forms

**Table 4 — Address forms accepted by Read and Write**

| Form | Example | Meaning |
|---|---|---|
| **File and element** | N7:0 | Word 0 of integer file 7. |
| **Element and bit** | B3:0/5 | Bit 5 of word 0 of binary file 3. |
| **File and bit number** | B3/17 | Bit 17 of binary file 3, counted from the start of the file. The same bit as B3:1/1. |
| **Structured field** | T4:0.ACC | The accumulated value of timer 0. PRE, ACC, DN, TT and EN are the usual fields. |
| **Array** | N7:0{5} | Five consecutive words starting at N7:0, returned as a list. Read only. |
| **Status file** | S:6 | Word 6 of the status file. Readable and, with care, writable. |

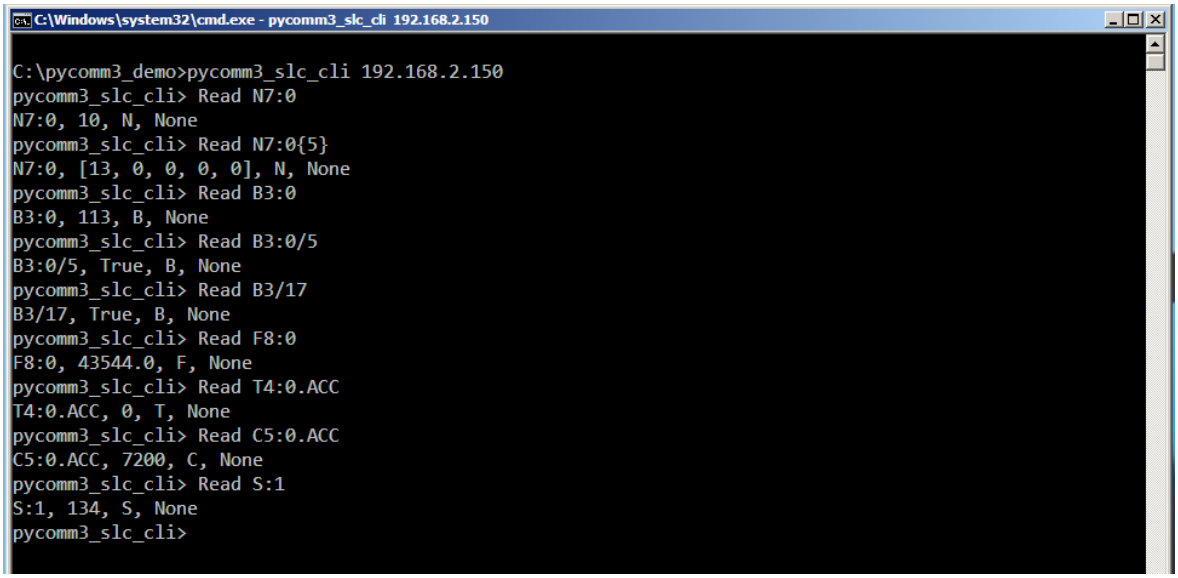
## 5.2 Data file types

The type letter in an address decides how the controller returns the value and what the application will let you write to it.

**Table 5 — Data file types in common use**

| Letter | File | Values |
|---|---|---|
| **O, I** | Output, Input | Integers; the physical I/O image. |
| **S** | Status | Integers; processor status and the clock. |
| **B** | Binary | Integers, usually addressed bit by bit. |
| **T** | Timer | Structured; PRE and ACC are integers. |
| **C** | Counter | Structured; PRE and ACC are integers. |
| **R** | Control | Structured; LEN and POS are integers. |
| **N** | Integer | Signed 16-bit integers. |
| **F** | Floating point | IEEE floats; the only type that accepts a fractional value. |

**Figure 3 — Reading each of the common address forms in a single console session**




**Note** A value read from a running controller is a snapshot taken during one scan. Two words read by two separate commands are not guaranteed to be consistent with one another. Where consistency matters, read the words as one array, as in N7:0{5}.

## 6. Output formats

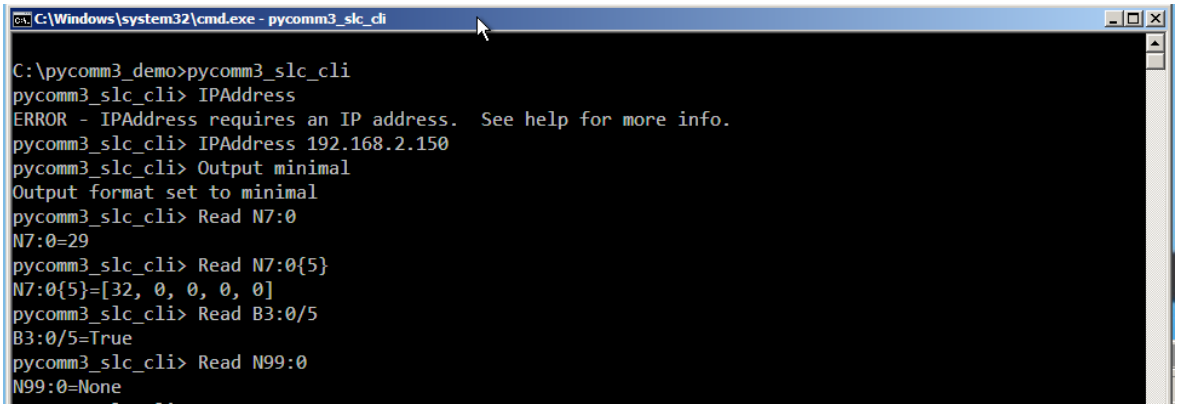
The Output command selects one of three formats. The format is a setting of the running session, so it is only useful in a console session; a single command run from the command line exits before a second command could use it. The default is raw.

**Table 6 — What each output format produces**

| Command | raw (default) | readable | minimal |
|---|---|---|---|
| **Read** | The whole result: address, value, file type and error. | The value on its own. | The address and the value, as tag=value. |
| **GetPLCTime** | Six raw results, one per clock word. | M/D/YYYY H:MM:SS | YYYY-MM-DD HH:MM:SS |
| **GetFaultCodes** | The full description of each code. | The same as raw. | One line per code. |
| **Everything else** | Unaffected. | Unaffected. | Unaffected. |

> **Note** Use readable when a person is reading the output and minimal when a script is. The minimal form of Read names the address alongside the value, which is the same tag=value form the tag file commands produce, so one parser handles both.

**Figure 4 — The minimal output format applied to Read, and the error IPAddress reports when it is given no address**



# 7. Command reference

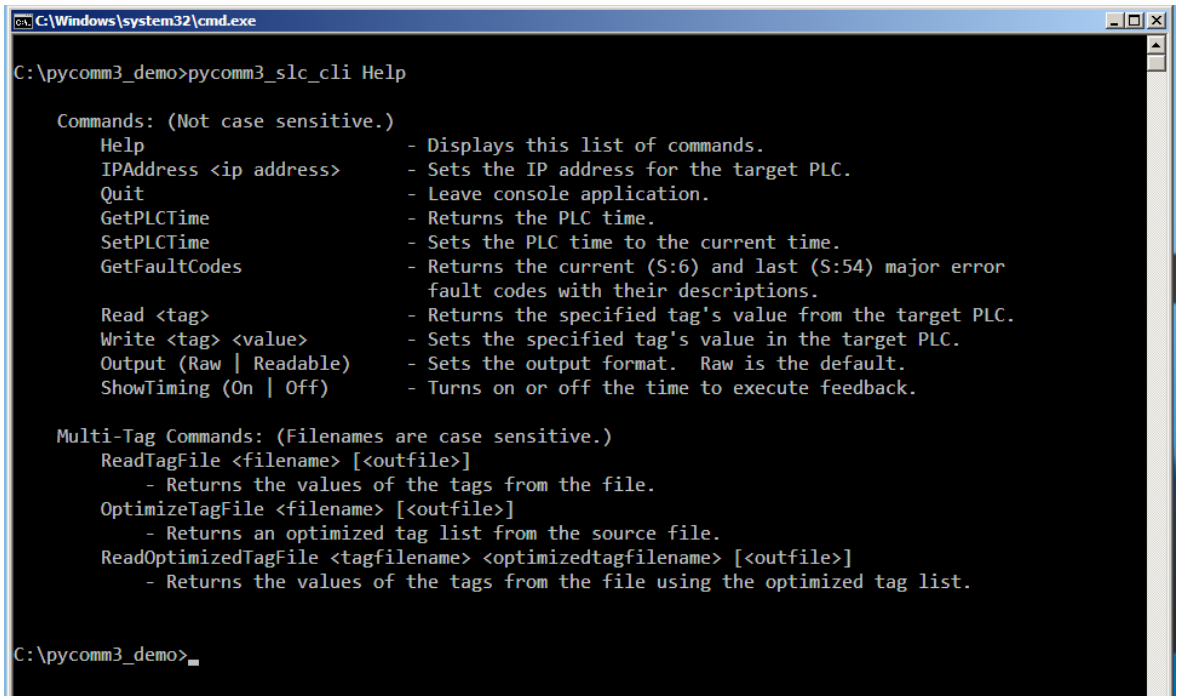
The commands are listed in the order the Help command prints them. Each description gives the syntax, what the command does, and what it does when something goes wrong.

## 7.1 Help

**`Help`**

Prints the list of commands and their arguments. Help needs no connection and can be run from the command line or at the prompt.

**Figure 5 — The Help command**



## 7.2 IPAddress

**`IPAddress <ip address>`**

Sets the target controller and opens the connection immediately. Any connection already open is closed first, so the command is also how you move a console session from one controller to another. If the connection cannot be opened, the reason is printed and the session is left with no controller.

```
pycomm3_slc_cli> IPAddress 192.168.2.150
pycomm3_slc_cli> Read N7:0
N7:0, 27, N, None
```

When a failure is reported, the message gives the underlying cause rather than the generic wrapper, so a wrong address, a blocked port and a refused connection read differently:

```
pycomm3_slc_cli> IPAddress 192.168.2.199
ERROR - Unable to connect to 192.168.2.199.  timed out
```

> **Note** IPAddress typed with no address is reported as an error and leaves the session and its existing connection untouched.

## 7.3 Quit

**`Quit`**

Leaves the console session and closes the connection. It has no effect from the command line, where the application exits on its own after one command.

> **Note** Quit must be the whole line. A trailing space makes the line something other than quit, and it is reported as an unrecognized command instead of ending the session.

## 7.4 Read

**`Read <tag>`**

Reads one address from the controller and prints the result. The address may be a word, a bit, a field of a structured element or an array, as described in Section 5.

In the default raw format the result is printed as four comma separated fields: the address as you typed it, the value, the data file type letter, and the error, which is None when the read succeeded.

```
C:\pycomm3_demo>pycomm3_slc_cli 192.168.2.150 Read N7:0
N7:0, 20, N, None

C:\pycomm3_demo>pycomm3_slc_cli 192.168.2.150 Read N7:0{5}
N7:0, [13, 0, 0, 0, 0], N, None
```

A read of an address that does not exist is reported in the error field rather than as a failure, and the application still exits with code 0:

```
C:\pycomm3_demo>pycomm3_slc_cli 192.168.2.150 Read N99:0
N99:0, None, N, Illegal Command or Format, Address may not exist or not
enough elements in data file
```

> **Note** The raw format abbreviates a long array with an ellipsis after six elements, because it prints the result object as the library formats it. The readable and minimal formats print every element, so switch format rather than splitting the read when you need the whole block.

## 7.5 Write

**`Write <tag> <value>`**

> **WARNING** Writing to a controller that is running equipment can move machinery immediately. Read Section 2 before using this command.

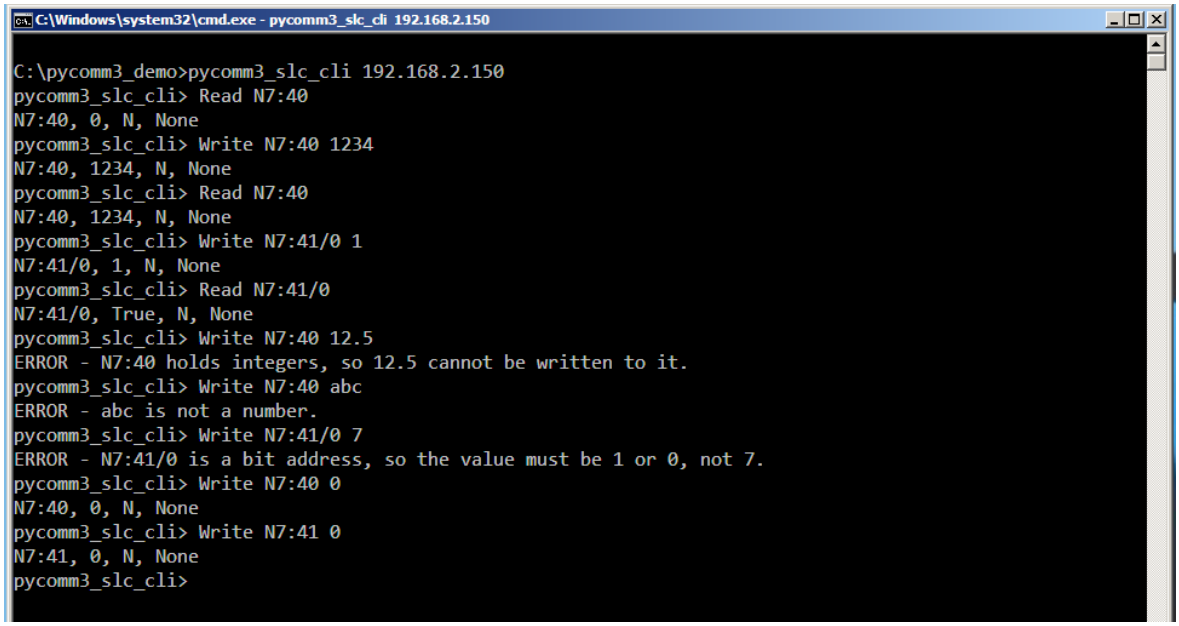
Writes one value to one address. The value is converted to match the data file before it is sent, and a value that cannot belong to that data file is refused before anything is written:

**Table 7 — How the value is interpreted for each kind of address**

| Address | Accepted values | Example |
|---|---|---|
| **A bit, such as B3:1/0 or N7:41/0** | 1 or 0. The words on, off, true, false, set and clear are also accepted. | Write B3:1/0 1 |
| **A floating point file, such as F8:4** | Any number, whole or fractional. | Write F8:4 12.5 |
| **Any other file, such as N7:40 or T4:0.PRE** | A whole number. A fractional value is refused rather than rounded. | Write N7:40 1234 |

The result is printed in the same four field form as a read, and reading the address back confirms the new value:

**Figure 6 — Writing a word and a bit, and the three values the command refuses**




> **Note** An address the controller program writes every scan will be overwritten by the program almost immediately. If a value you write does not appear to stick, the program is writing it too.

## 7.6 GetPLCTime

**`GetPLCTime`**

Reads the controller clock, which occupies the status words S:37 through S:42 as year, month, day, hour, minute and second. What is printed depends on the output format, as set out in Section 6. The raw format reads the six words individually and prints each result; the readable and minimal formats read all six in one operation and print a formatted date and time.

```
pycomm3_slc_cli> Output readable
Output format set to readable
pycomm3_slc_cli> GetPLCTime
11/10/2026 8:08:33
```

> **Note** ShowTiming has no effect on GetPLCTime or SetPLCTime. Neither command reports how long it took.

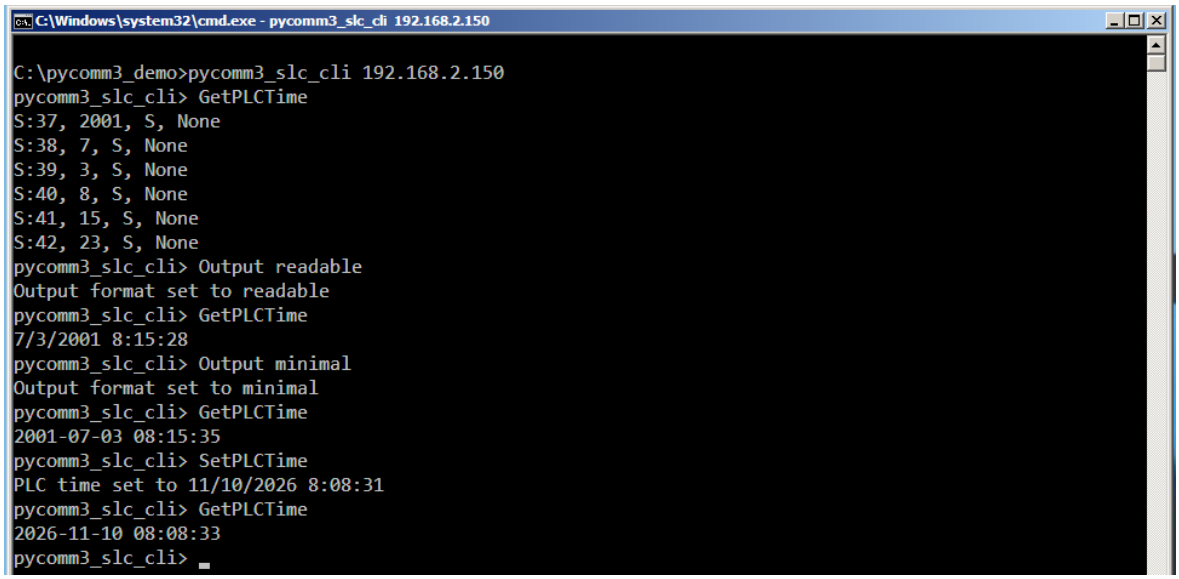
## 7.7 SetPLCTime

**`SetPLCTime`**

> **WARNING** This command changes the controller clock to the clock of the PC. Check the PC clock first, and read Section 2.4 for what a clock change can do to a running program.

Writes the current local date and time of the PC into S:37 through S:42 and prints the time it wrote. In the figure below the controller clock had drifted to the year 2001; GetPLCTime shows it in each of the three output formats, SetPLCTime corrects it, and GetPLCTime confirms the result.

**Figure 7 — Reading a controller clock that had drifted, correcting it with SetPLCTime, and reading it back**




> **Note** The confirmation line reports the time the application sent, not a value read back from the controller. The write is not verified. Follow a SetPLCTime with a GetPLCTime when it matters.

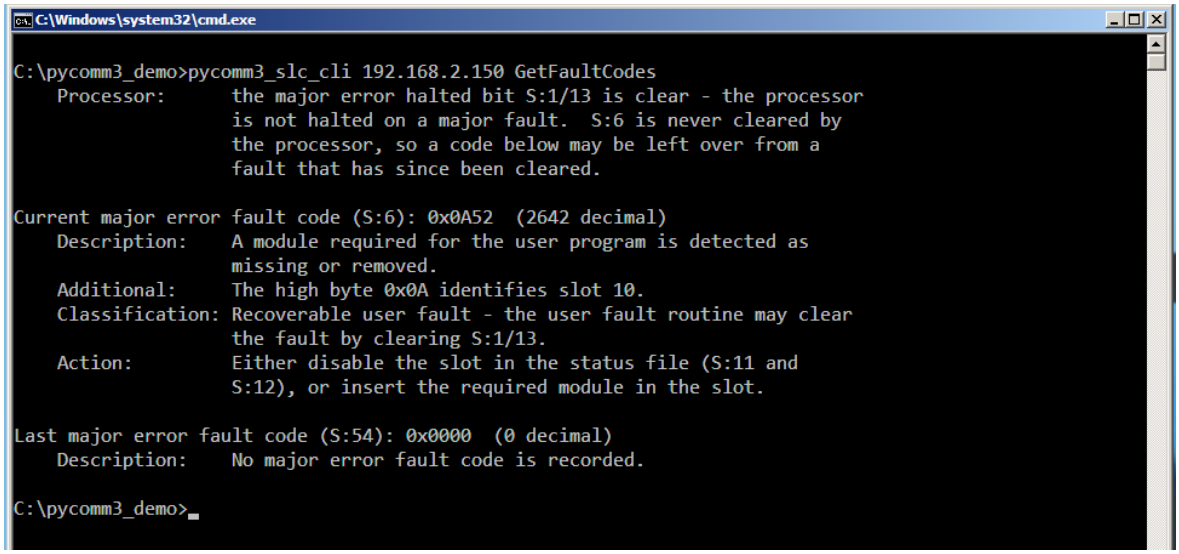
## 7.8 GetFaultCodes

**`GetFaultCodes`**

Reports the current major error fault code in S:6 and the last major error fault code in S:54, each with the Rockwell description of what it means, how it is classified and what to do about it. It also reports the major error halted bit S:1/13, which is what tells you whether the processor is actually halted on the code being reported.

This distinction matters because the processor never clears S:6. A code in S:6 with S:1/13 clear is the record of a fault that has already been dealt with, not a fault happening now.

**Figure 8 — GetFaultCodes reporting a module fault, with the slot number taken from the high byte of the code**

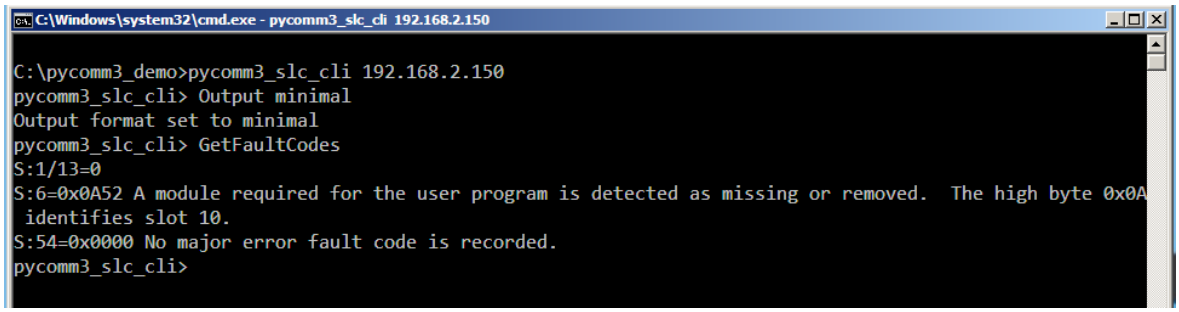



Where a code carries extra information the application interprets it rather than leaving you to decode it:

- For the I/O fault codes the high byte is the slot number, reported as part of the description. 0x1F means the slot could not be determined.
- For code xx39 the high byte is the data file number.
- For code 0020 the minor error bits in S:5 are read and the bit that caused the fault is named.
- For code 00A1 on an SLC 5/05 the Ethernet daughterboard code in S:15 is read and reported.

With the output format set to minimal the same information is reduced to one line per code, which suits a log file or a script that parses the output:

**Figure 9 — The same fault reported in the minimal output format**



**Note** S:54 exists only on SLC 5/03 and higher processors. On an earlier processor the last fault code line reports that the word is unavailable, which is expected rather than a failure.

## 7.9 Output

**`Output <raw | readable | minimal>`**

Selects the output format for the rest of the session, as described in Section 6. The default is raw. The format argument must be lower case; anything else is rejected with the message Invalid output format.

```
pycomm3_slc_cli> Output readable
Output format set to readable
pycomm3_slc_cli> Output Readable
Invalid output format
```

## 7.10 ShowTiming

**`ShowTiming <on | off>`**

Turns on a line reporting how long each command took. It is off by default. Like the Output argument, on and off must be lower case.

Timing is reported for Read, Write, GetFaultCodes, ReadTagFile, OptimizeTagFile and ReadOptimizedTagFile. It is not reported for GetPLCTime or SetPLCTime. The time covers the command only, not the time taken to open the connection, which is why the same read is much faster in a console session than as a single command from the command line.

```
pycomm3_slc_cli> ShowTiming on
ShowTiming set to on.
pycomm3_slc_cli> Read N7:0{5}
[41, 0, 0, 0, 0]
Executed in   0.078 seconds.
```

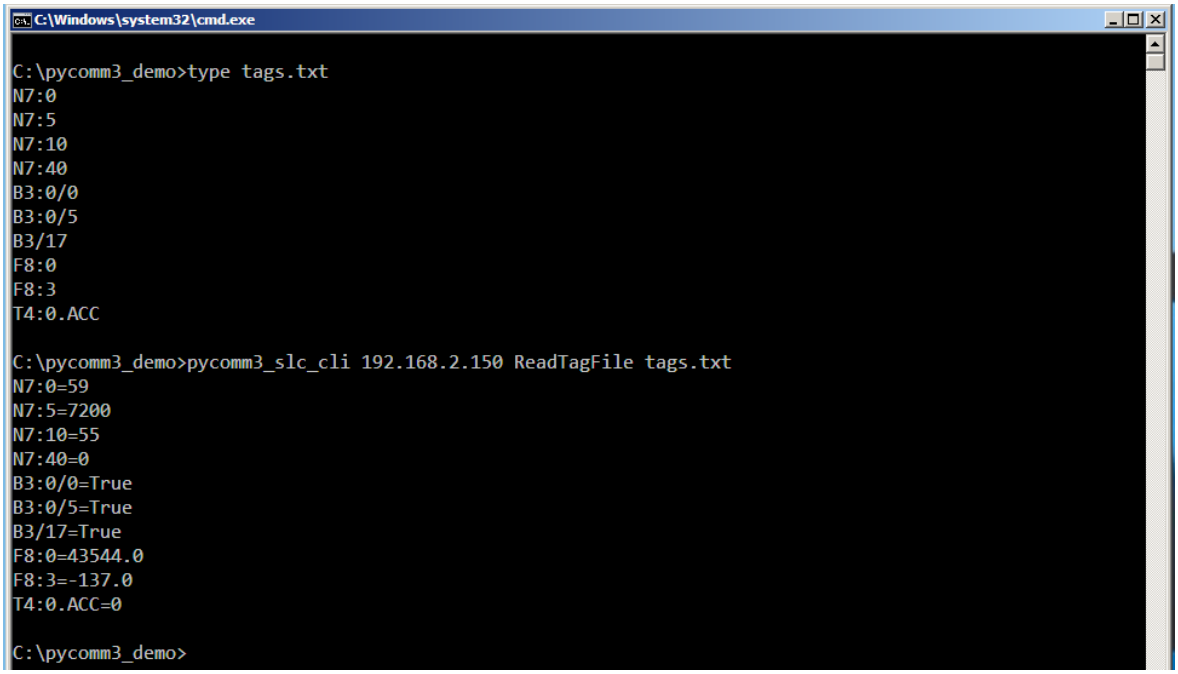
## 7.11 ReadTagFile

**`ReadTagFile <filename> [<outfile>]`**

Reads a list of addresses from a text file, one address per line, and prints each one as tag=value. Blank lines are ignored. Give a second file name and the result is written to that file instead of the screen.

Each address is read individually, so a file of a hundred addresses is a hundred round trips to the controller. OptimizeTagFile and ReadOptimizedTagFile exist to avoid that cost; see Section 7.13.

**Figure 10 — A tag list file and the values ReadTagFile returned for it**




An address that cannot be read does not stop the run. It is reported in place, so a single bad line in a long list is easy to find. An address the controller rejects reports None, and an address whose read raised an error, such as one attempted after the connection dropped, reports !ERROR!:

```
N7:0=34
B3:0/0=True
N99:0=None
F8:0=43544.0
```

> **Note** The output format set by the Output command does not affect ReadTagFile. Its output is always tag=value.

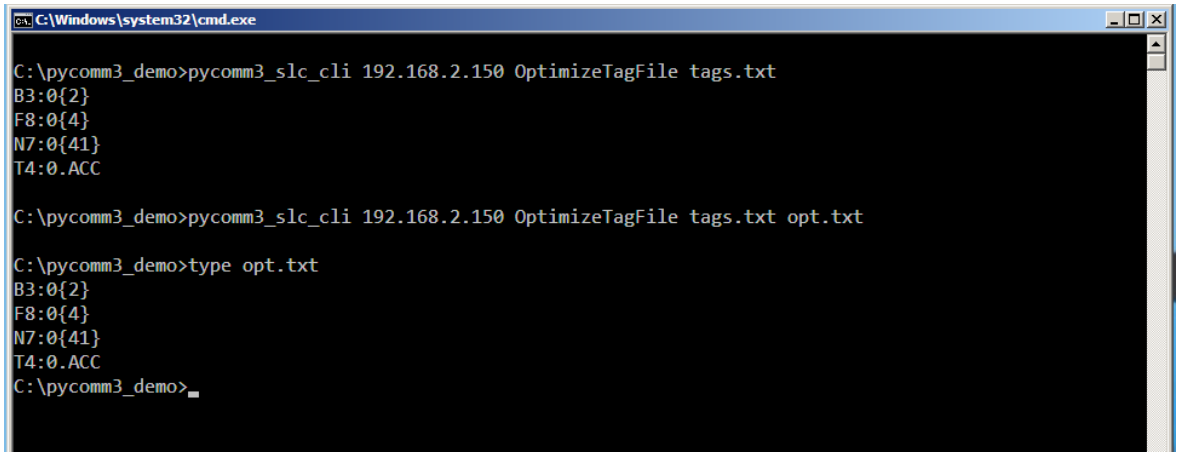
## 7.12 OptimizeTagFile

**`OptimizeTagFile <filename> [<outfile>]`**

Turns a list of individual addresses into a much shorter list of array reads that covers all of them. It is the only command that does not need a controller connection, because it works entirely on the text file.

The command sorts the list, converts any address written as a file and bit number into word and bit form, then for each data file works out the lowest and the highest word used and emits an array read spanning them.

**Figure 11 — Ten individual addresses reduced to four reads**




Two limits apply to the arrays it produces:

- Integer and binary files are split into blocks of at most 120 words.
- Floating point files are split into blocks of at most 60 elements.

An address whose element part is not a plain number, such as T4:0.ACC, cannot be grouped and is carried through to the optimized list unchanged.

> **WARNING** The optimized list covers the whole span between the lowest and the highest word used in each data file, including words that were never in your list. For reading this is only a question of how much data crosses the network, but a list containing N7:0 and N7:200 produces a read of 201 words even if you wanted two.

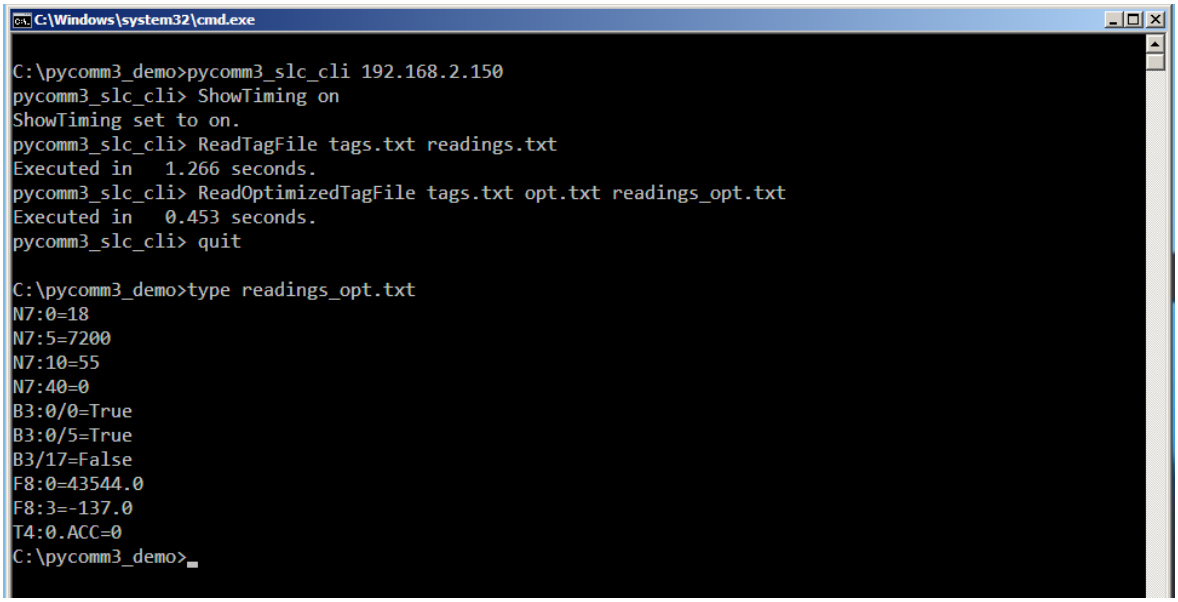
## 7.13 ReadOptimizedTagFile

**`ReadOptimizedTagFile <tagfilename> <optimizedtagfilename> [<outfile>]`**

Reads the blocks listed in the optimized file, then picks out of them the value of every address in the original tag file and prints the result in the same tag=value form as ReadTagFile. Bit addresses are extracted from the word that contains them, so a bit costs no extra round trip. All three arguments are positional and both input files are required.

The saving grows with the length of the list. In the session below, ten addresses took 1.266 seconds read one at a time and 0.453 seconds read through the optimized list, and both commands wrote their results to a file:

**Figure 12 — The same ten addresses read individually and through the optimized list, with the results written to a file**




Two values in the output deserve attention:

- **=NONE** NONE means the address was not covered by any block in the optimized file. Rebuild the optimized file from the tag file whenever you change the tag file.
- **=!ERROR!** ERROR means the read of the block containing that address raised an error, so every address in that block reports the same.

**WARNING** Two kinds of tag list make this command fail with an unhandled error rather than a message. A list in which a floating point file contributes only one address produces a single element block that the command cannot parse, and a list containing an address the controller rejects produces a block the command cannot parse either. Both are described in Section 9. Confirm a new tag list with ReadTagFile before you rely on it here.

The normal sequence is to build the optimized file once and then reuse it:

```
pycomm3_slc_cli> OptimizeTagFile tags.txt opt.txt
pycomm3_slc_cli> ReadOptimizedTagFile tags.txt opt.txt readings.txt
```

**Note** Output files are written with line feed line endings and no newline at the end of the last line. They open correctly in Excel, in a text editor such as Notepad++ and in a script, but Windows Notepad shows the whole file as a single line.

## 8. Error messages and troubleshooting

Errors are printed to the console and, except for a failure to connect in single command mode, do not change the exit code.

**Figure 13 — The four most common errors, and the exit code each leaves behind**

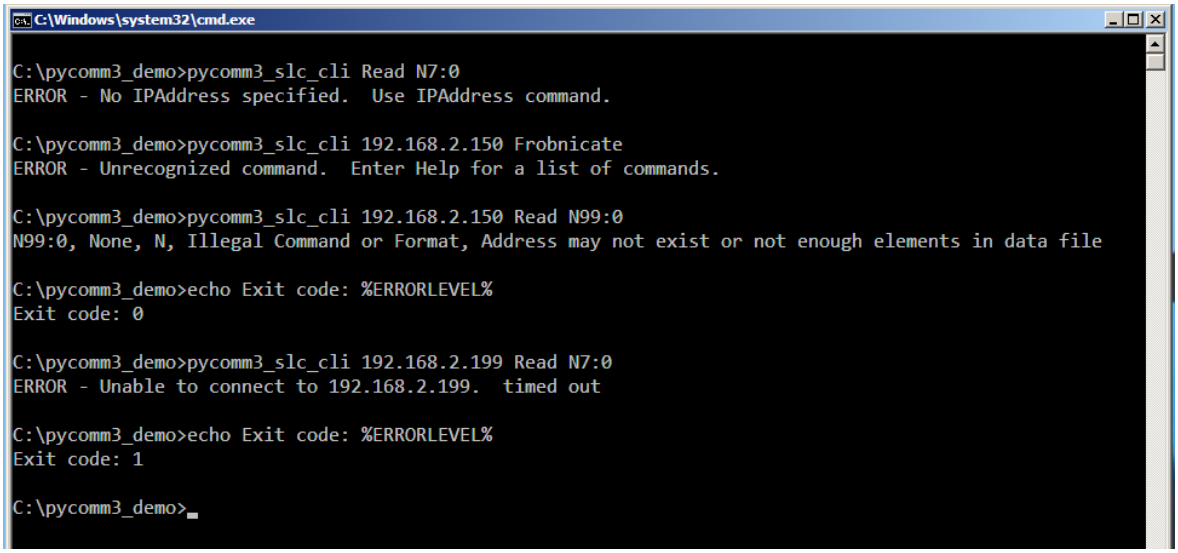



**Table 8 — Error messages and what to do about them**

| Message | Cause and remedy |
|---|---|
| **ERROR - No IPAddress specified.** | The command needs a controller and none is set. Supply the address on the command line, or use IPAddress. |
| **ERROR - Unable to connect to <address>.** | No connection could be opened. The text after the address is the underlying reason: timed out means nothing answered, refused means something answered but not on the EtherNet/IP port. Check the address, the cabling and the firewall. |
| **Illegal Command or Format, Address may not exist** | The controller rejected the address. The data file may not exist, or may be shorter than the element you asked for. |
| **ERROR - Unrecognized command.** | The first word is not a command. Run Help for the list. A Quit with a trailing space produces this message. |
| **ERROR - Error opening the file <name>.** | The tag file could not be opened. Check the name and that you are in the folder that holds it. |
| **ERROR - Invalid number of arguments.** | ReadOptimizedTagFile needs both the tag file and the optimized file. |
| **Invalid output format** | The argument to Output must be raw, readable or minimal, in lower case. |
| **Invalid ShowTiming argument.** | The argument to ShowTiming must be on or off, in lower case. |

| Message | Cause and remedy |
|---|---|
| **ERROR - <value> is not a number.** | Write was given a value that is not numeric. |
| **ERROR - <tag> holds integers.** | A fractional value was written to a file that holds whole numbers. Write it to a floating point file, or round it yourself. |
| **ERROR - <tag> is a bit address.** | A bit takes 1 or 0 only. |
| **ERROR - Write requires a tag and a value.** | Write was given only one argument. |

### 8.1 If nothing connects at all

- Ping the controller from the same PC. If the ping fails the problem is the network, not the application.
- Check that the PC is on the same subnet as the controller, or that a route exists.
- Check that nothing else holds an exclusive session with the controller.
- Confirm that the controller has an Ethernet interface that speaks EtherNet/IP. A DH+ or serial only processor cannot be reached directly.

## 9. Known limitations

The behaviors below are present in version 0.3.0. They are listed so that they are not mistaken for a fault in the controller or the network.

**Table 9 — Known limitations in version 0.3.0**

| Area | Behavior |
|---|---|
| **ReadOptimizedTagFile with a single floating point address** | A tag list in which a floating point file contributes only one address makes the command fail with an unhandled error. Two or more addresses in that file are handled correctly. |
| **ReadOptimizedTagFile with a rejected address** | A tag list containing an address the controller rejects makes the command fail with an unhandled error. ReadTagFile reports the same address as None and continues. |
| **Quit with trailing whitespace** | Is not recognized as Quit and is reported as an unrecognized command. |
| **Case of Output and ShowTiming arguments** | Must be lower case, although command names themselves are not case sensitive. |
| **Output file line endings** | Line feed only, with no newline after the last line. Windows Notepad shows such a file as one line. |
| **SetPLCTime verification** | The confirmation line reports what was sent, not what the controller holds. The write is not read back. |
| **OptimizeTagFile span** | Covers every word between the lowest and the highest used in each data file, including words not in the list. |
| **32-bit Windows** | Not supported. The executable is 64-bit. |
| **Host names** | Only a dotted numeric address is reliable as the first argument. |

## Appendix A — Command summary

**Table 10 — All commands, in the order Help lists them**

| Command | Purpose | Needs a controller |
|---|---|---|
| **Help** | List the commands. | No |
| **IPAddress <ip address>** | Set the target and connect. | No |
| **Quit** | Leave the console session. | No |
| **GetPLCTime** | Read the controller clock. | Yes |
| **SetPLCTime** | Set the controller clock from the PC. | Yes |
| **GetFaultCodes** | Report the major error fault codes. | Yes |
| **Read <tag>** | Read one address. | Yes |
| **Write <tag> <value>** | Write one address. | Yes |
| **Output <format>** | Select raw, readable or minimal. | No |
| **ShowTiming <on \| off>** | Report execution time. | No |
| **ReadTagFile <filename> [<outfile>]** | Read every address in a file. | Yes |
| **OptimizeTagFile <filename> [<outfile>]** | Reduce a list of addresses to array reads. | No |
| **ReadOptimizedTagFile <tagfilename> <optimizedtagfilename> [<outfile>]** | Read a list through its optimized form. | Yes |

## Appendix B — Reference documents

- Allen-Bradley SLC 500 Instruction Set Reference Manual, publication 1747-RM001G-EN-P, November 2008. Chapter 16, Troubleshooting Faults, and Appendix B, SLC Status File. The source of every fault description printed by GetFaultCodes.
- The pycomm3 library and its documentation, at https://github.com/ottowayi/pycomm3. pycomm3_slc_cli is a command line wrapper around its SLCDriver.
- The pycomm3_slc_cli repository and releases, at https://github.com/AppendAutomation/pycomm3_slc_cli.

pycomm3_slc_cli is released under the MIT License. With thanks to ruscito, ottowayi and the contributors who make pycomm3 possible.